CS x290/ECE x100
Conte

Instruction-Level Parallelism (ILP)

Copyright © 1995-2021 T. M. Conte

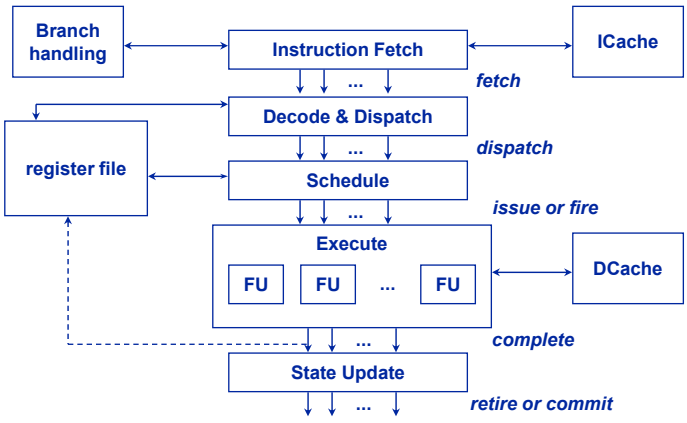
1

Limits of Pipelining

- CPU_time = IC x CPI x CT
- Scalar pipelining
  - CPI ≥ 1 (Flynn's bottleneck)
  - ∴ reduce CT further, increase instructions per *second*
- Problem: limits of (single instruction per cycle) pipelining
  1. Continue to reduce CT, but can only break logic up so far
  2. Latch overhead and clock skew start to dominate
     - Overhead becomes a larger component of cycle
     - Pipelining eventually becomes inefficient
  3. Deep pipelines are ineffective when there are hazards

2

View of superscalar processor



ILP-5

5

Roadmap for this Chapter

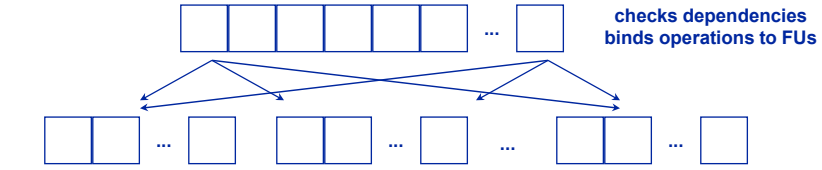
- Dispatch and Schedule hardware algorithms
- State update designs
- Instruction fetch designs
- Doing it all in a completely different way…(more details to follow)

ILP-6

6

Dispatch and scheduling

- Dispatch and schedule units maintain queueues:



entries for:
function unit #1 *function unit #2* *function unit #N*

ILP-7

7

A little about function(al) units

- Function(al) units = FUs
- Typical latencies and why:

Type	Latency	Why
Integer ADD	1	The ALU is critical
FP MUL	3	Stage 1: Denormalize; Stage 2: add exp, multi mantissas; Stage 3: Normalize
FP ADD	2	Stage 1:Denormalize; Sage 2: Add and normalize (1 cycle)
Load (LW)	2	The cache pipeline
Store (SW)	1	Store buffering

ILP-8

8

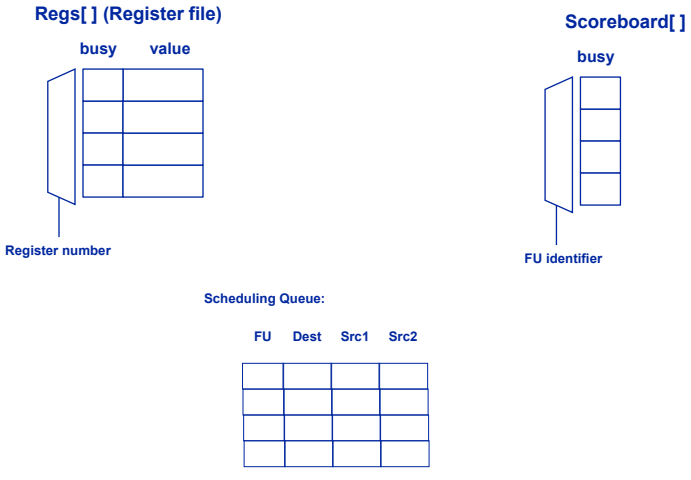
A classification of approaches

- Fire (issue) (F) either in-order (I) or out-of-order (O)
- Complete either in-order (I) or out-of-order (O)
- So FICI are pipelines like the 5-stage pipeline from last section
- The Cray-1 (simple interlocking) was FICO
- The IBM 360/91 was FOCO
- (Note "FOCI" isn't a useful configuration!)

ILP-10

10

A dynamic scheduling algorithm: Simple interlocking (FICO) Hardware view--



ILP-13

13

A dynamic scheduling algorithm: Simple interlocking (FICO)

```
1. [Dispatch/Scheduling unit (combined unit)]
      for each Inst in SchedQueue do:
            if ( !Regs[Inst.Src1].Busy
                  AND !Regs[Inst.Src2].Busy
                  AND !Regs[Inst.Dest].Busy
                  AND !Scoreboard[Inst.FU].Busy )
            then
                  Scoreboard[Inst.FU].Busy = True
                  Regs[Inst.Dest].Busy = True
                  // Issue instruction to function unit Inst.FU
            else
                  exit loop          // halt issuing

2. [Execution unit, at completion of instruction Inst]
      (a) Regs[Inst.Dest].Busy = False
      (b) if (Inst.FU pipeline advances and now first stage is free) then
            Scoreboard[Inst.FU].Busy = False
```


14

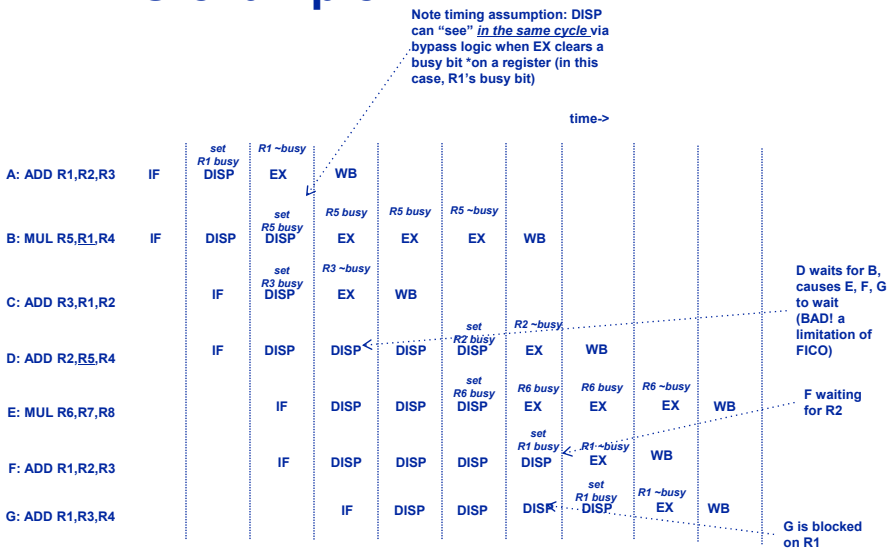
A dynamic scheduling algorithm: Simple interlocking (FICO) (simplified)

```
1. [Dispatch/Scheduling unit (combined unit)]
      Scan SchedQueue in FIFO order
      For each Inst in SchedQueue:
            if ( Both source regs of Inst are not busy
                  AND destination reg is not busy
                  AND scoreboard says FU is not busy )
            then
                  Mark FU busy in Scoreboard
                  Dest Inst's dest reg as busy
                  // Issue Inst
            else
                  exit loop          // halt issuing

2. [Execution unit, at completion of instruction Inst]
      (a) Clear busy bit of dest reg of Inst
      (b) if (Inst.FU pipeline advances and now first stage is free) then
            clear the scoreboard bit for Inst.FU
```


15

FICO example



16

How FICO enforces dependencies

- **Flow dependence**

Scheduling/dispatch queue

R1.Busy set = 1

A: ADD R1, R2,R3

B: ADD R4, R1, R3

R1.Busy checked: halt firing

scanned in program order

- **Anti-dependence**

Scheduling/dispatch queue

R1's new value will be written after B's execution begins

A: ADD R4, R1, R3

B: ADD R1, R2, R3

R1's old value read during scheduling/dispatch

- **Output dependence**

Scheduling/dispatch queue

A: MUL R1, R2, R3

B: ADD R1, R4, R5

R1.Busy set = 1

R1.Busy checked: halt firing

17

Tomasulo's Algorithm

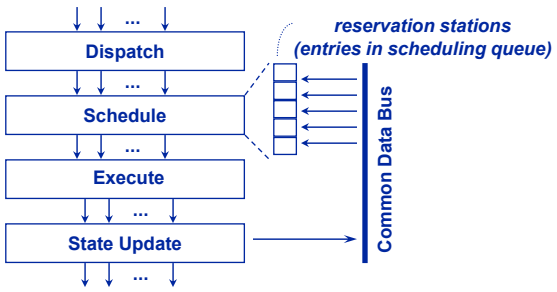
- Robert Tomasulo consulted on the IBM 360 model 91
  - Born of necessity: model 85 vs. 91 competition (the '91 lost…)
- Used in IBM 360/91 floating-point unit
  - Many long-latency operations
  - Need dynamic scheduling: mitigate long stalls
- ISA specified only 4 floating-point registers
  - Need register renaming: with only 4 registers, false dependencies pop up quickly
  - Renaming was a requirement!

ILP-20

20

Reservation stations and the Common Data Bus

- Scheduling queue entries become "reservation stations": places to park instructions waiting on specific values to be calculated
- Announcements of specific values are broadcast on the "Common Data Bus"
  - In original IBM 360 model 91, there was one CDB. Today there are often several (renamed a less awkward term: "result buses")



ILP-21

21

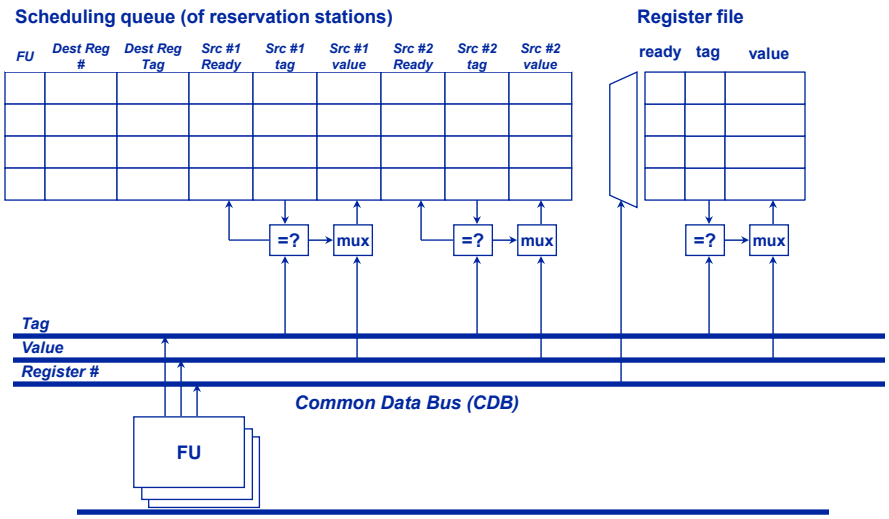
Renaming registers

- Problem: anti, output dependencies can cause many cycle stalls (recall IPC ~ # regs)
- Solution: Take advantage of reservation stations and common data bus to tag values with new "names" different from register #'s
  - Assign "tags" (new names) to destination register names in program order (during Dispatch)
  - Add "tag" fields to reservation stations, common data bus, register file
- Note well: renaming registers and out-of-order issue are separate capabilities, although they are often combined

ILP-22

22

Tomasulo Hardware View:



ILP-23

23

High level Tomasulo agorithm

- Dispatch: For each inst, put in sched queue: looks up src regs for value or tag, assigns a tag to dest reg and marks it not ready
- Scheduling: manages the sched queue: listens to bus to update ready bits/values, fires insts in sched queue that have both source regs ready and FU free
- Complete: broadcasts result on bus, frees slot in sched queue
- Reg file: if bus tag matches reg entry, updates value and marks reg ready

24

Tomasulo algorithm, part 1

For each cycle of execution…

```
[Dispatch unit ] for all instructions I in DispQ do:
    if (SchedQ is not full) then
        (a) Add I to first free slot of SchedQ  (="RS")      // Call this RS (for "reservation station") below
        (b) Delete I from DispQ                               // Insttruction I is now "dispatched"
        (c) RS.FU = I.FU
        (d) RS.Dest = I.Dest
        (e) for i = 0..1 do
                if (Regs[I.src[i]].Ready = True) then
                    RS.Src[i].Value = Regs[I.src[i]].Value
                    RS.Src[i].Ready = True
                else
                    RS.Src[i].Tag = Regs[I.src[i]].Tag  // Copy the tags from the RF
                    RS.Src[i].Ready = False
        (f) Regs[I.Dest].Tag = unique tag id    // tag (rename) the destination
        (g) RS.DestRegTag = Regs[I.Dest].Tag
        (h) Regs[I.Dest].Ready = False                        // value ready only after execution

    else exit loop // stop dispatching if scheduling queue is full
```


25

Tomasulo algorithm, part 2

```
[ Scheduling unit ]
for each RS = entry in SchedQueue do:
      (a) for i = 0..1 do:
            if (CDB.Tag = RS.Src[i].Tag) then          // broadcast on CDB matches stored tag
                  RS.Src[i].Ready = True
                  RS.Src[i].Value = CDB.Value
      (b) if (RS.Src[0].Ready = True AND RS.Src[1].Ready = True ) // WAKEUP: If all registers are
ready...
            and Scoreboard[RS.FU].Busy = False) then
                  Scoreboard[RS.FU].Busy = True // ...reserve the FU and issue
                  Issue the instruction at RS on FU
```

step (b) is called "wake-up"
when more than one instruction is ready, picking between them is called "select" (here we're using FIFO as a select algorithm)

ILP-26

26

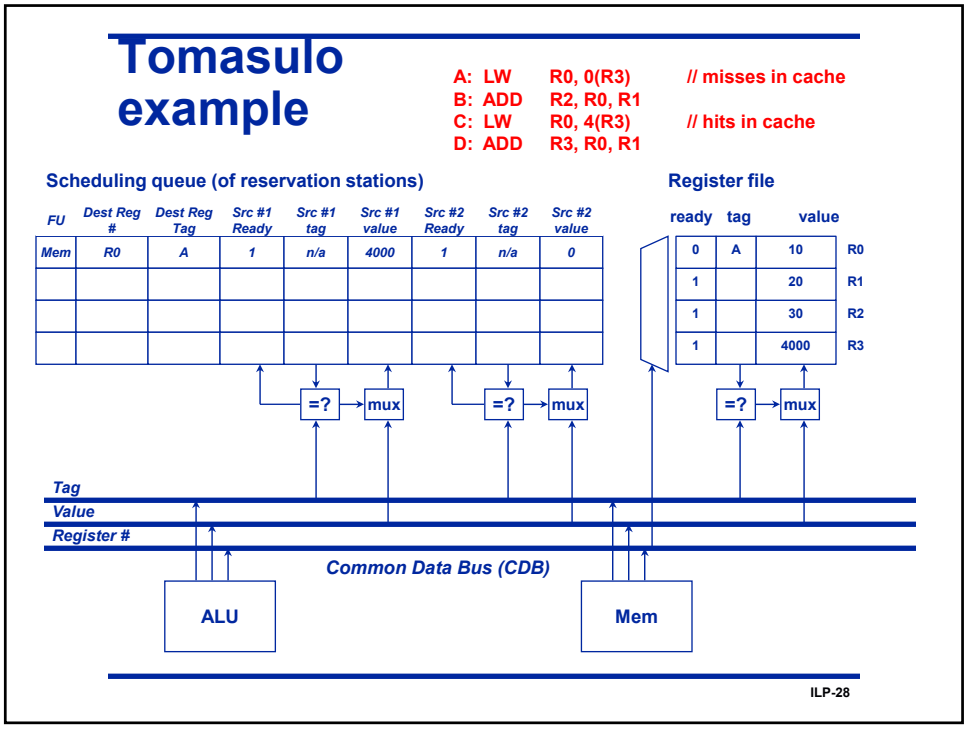
Tomasulo Algorithm, part 3

```
[ Execution unit, for function unit FU]
      if (FU pipeline advances and now first stage is free) then Scoreboard[FU].Busy = False
[ Execution unit, at completion of instruction I ]
      if (CDB.Busy = False) then                   // Broadcast the instruction results
            (a) CDB.Busy = True
            (b) CDB.Tag = I.Tag
            (c) CDB.Value = I.Value
            (d) CDB.Reg = I.Dest
            (e) if (I.FU is not pipelined) then
                  Scoreboard[I.FU].Busy = False
            (f) Delete I from Sched Queue

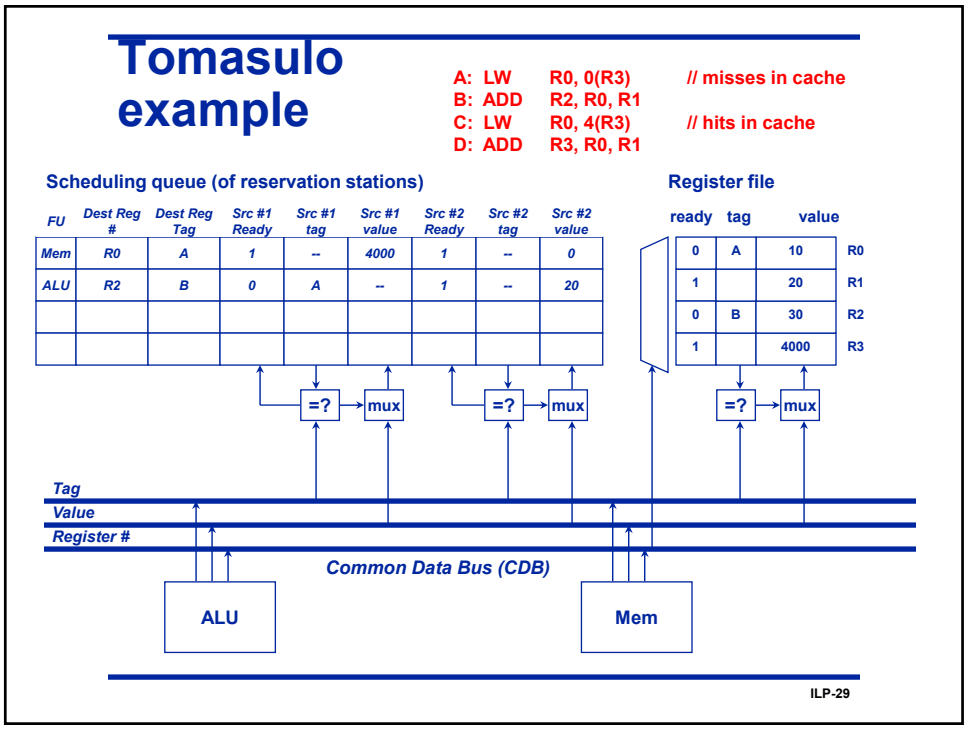
[ Register file ]
      if (CDB.Busy = True and
      CDB.Tag = Regs[CDB.Reg].Tag) then
            (a) Regs[CDB.Reg].Ready = True         // Update the RF contents
            (b) Regs[CDB.Reg].Value = CDB.Value
            (c) CDB.Busy = False
```

ILP-27

27


Tomasulo example
A: LW R0, 0(R3) // misses in cache
B: ADD R2, R0, R1
C: LW R0, 4(R3) // hits in cache
D: ADD R3, R0, R1
Scheduling queue (of reservation stations)
Register file
FU
Dest Reg #
Dest Reg Tag
Src #1 Ready
Src #1 tag
Src #1 value
Src #2 Ready
Src #2 tag
Src #2 value
Mem R0 A 1 n/a 4000 1 n/a 0
ready tag value
0 A 10 R0
1 20 R1
1 30 R2
1 4000 R3
=? mux
Tag
Value
Register #
Common Data Bus (CDB)
ALU
Mem
ILP-28

28


Tomasulo example
A: LW R0, 0(R3) // misses in cache
B: ADD R2, R0, R1
C: LW R0, 4(R3) // hits in cache
D: ADD R3, R0, R1
Scheduling queue (of reservation stations)
Register file
FU
Dest Reg #
Dest Reg Tag
Src #1 Ready
Src #1 tag
Src #1 value
Src #2 Ready
Src #2 tag
Src #2 value
Mem R0 A 1 -- 4000 1 -- 0
ALU R2 B 0 A -- 1 -- 20
ready tag value
0 A 10 R0
1 20 R1
0 B 30 R2
1 4000 R3
=? mux
Tag
Value
Register #
Common Data Bus (CDB)
ALU
Mem
ILP-29

29

Tomasulo example

A: LW R0, 0(R3) // misses in cache
B: ADD R2, R0, R1
C: LW R0, 4(R3) // hits in cache
D: ADD R3, R0, R1

Scheduling queue (of reservation stations)

FU	Dest Reg #	Dest Reg Tag	Src #1 Ready	Src #1 tag	Src #1 value	Src #2 Ready	Src #2 tag	Src #2 value
Mem	R0	A	1	--	4000	1	--	0
ALU	R2	B	0	A	--	1	--	20
Mem	R0	C	1	--	4000	1	--	4

Register file

	ready	tag	value
R0	0	C	10
R1	1		20
R2	0	B	30
R3	1		4000

=? mux =? mux =? mux

Tag
Value
Register #

Common Data Bus (CDB)

ALU Mem

ILP-30

30

Tomasulo example

A: LW R0, 0(R3) // misses in cache
B: ADD R2, R0, R1
C: LW R0, 4(R3) // hits in cache
D: ADD R3, R0, R1

Scheduling queue (of reservation stations)

FU	Dest Reg #	Dest Reg Tag	Src #1 Ready	Src #1 tag	Src #1 value	Src #2 Ready	Src #2 tag	Src #2 value
Mem	R0	A	1	--	4000	1	--	0
ALU	R2	B	0	A	--	1	--	20
Mem	R0	C	1	--	4000	1	--	4
ALU	R3	D	0	C	--	1	--	20

Register file

	ready	tag	value
R0	0	C	10
R1	1		20
R2	0	B	30
R3	0	D	4000

=? mux =? mux =? mux

Tag
Value
Register #

Common Data Bus (CDB)

ALU Mem

ILP-31

31

Tomasulo problems / solutions

- CDB limits # instruction that can complete per cycle to one instruction
  - Solution: Have N CDBs. Called "result buses" because they are no longer "common" data buses. Now can complete N instructions per cycle
- The scheduling queue needs a fully associative lookup for each result bus
  - Very expensive!
  - Solution…

ILP-32

32

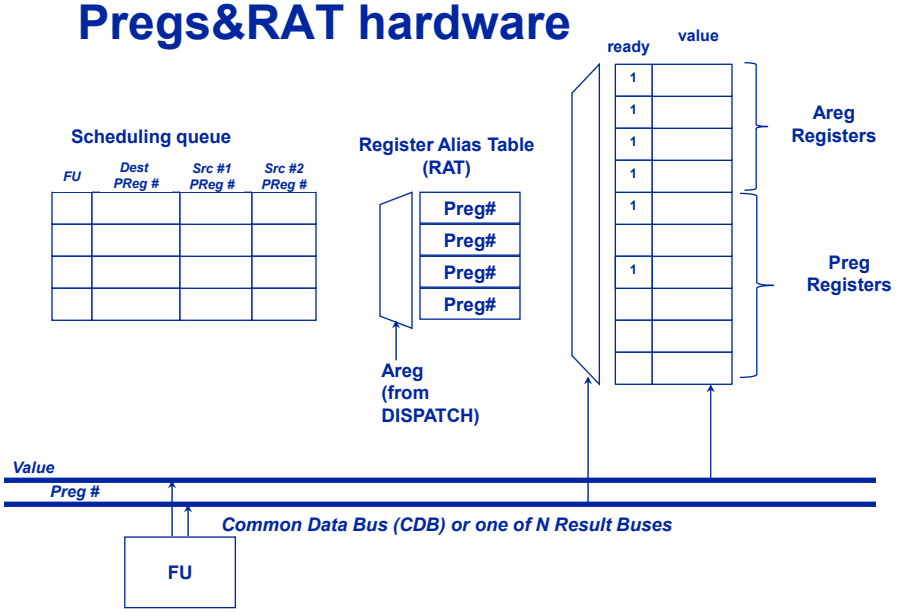
Alternative Implementation: Physical regs and RAT

- Instead of TAGs, use real registers
  - Extend the register file with and PHYSICAL (UNARCHITECTED) registers (Pregs) in addition to traditional (architected) registers, called "Aregs"
  - (Architected– ie, traditional– registers are called "Aregs")
  - Use a TABLE to map most recent Preg to Aregs – called the Register Alias Table (RAT)

ILP-33

33

Pregs&RAT hardware



ILP-34

34

PRegs&RAT

- At DISPATCH of instruction "*I*"
  - Say D = the Areg dest register of *I*: pick an unused Preg index *"P"*
  - Set RAT[D] = P
- At scheduling unit for entry *RS*:
  - Wake up *RS* if register for each source of *RS* is ready
  - Scheduler (or FU) reads the register file for value
- On instruction completion of *I*:
  - Write the result into Preg in the physical register file part and mark it as "Ready"
- The Areg portion of the register file is used when freeing Pregs (details to follow…)

ILP-35

35

Tomasulo with Pregs&RAT, part 1

For each cycle of execution…

```
[Dispatch unit ] for all instructions I in DispQ do:
      if (SchedQ is not full AND there are free Pregs) then
            (a) Add I to first free slot of SchedQ  (="RS")      // Call this RS (for "reservation station") below
            (b) Delete I from DispQ                               // Instruction I is now "dispatched"
            (c) RS.FU = I.FU
            (d) for i = 0..1 do
                  RS.SRC[i] = RAT[I.srci]
            (e) RAT[I.Dest] = Find a free Preg #   // rename the dest
            (f) RS.Dest_Preg_index = RAT[I.Dest]
            (g) Regs[ RAT[I.Dest] ].Ready = False
      else exit loop // stop dispatching if scheduling queue is full or there are no free PRegs


[ Scheduling unit ]
for each RS = entry in SchedQueue do:
      if (Regs[ RS.SRC[0] ].Ready = True AND
         Regs[ RS.SRC[1] ].Ready = True ) // WAKEUP: If all registers are ready...
            and Scoreboard[RS.FU].Busy = False) then
                  Scoreboard[RS.FU].Busy = True // ...reserve the FU and issue
                  Issue the instruction at RS on FU
```


36

Tomasulo with Pregs&RAT part 2

```
[ Execution unit, for function unit FU]
      if (FU pipeline advances and now first stage is free) then Scoreboard[FU].Busy = False
[ Execution unit, at completion of instruction I ]
      if (CDB.Busy = False) then                          // Broadcast the instruction results
            (a) CDB.Busy = True
            (b) CDB.Value = I.Value
            (c) CDB.PReg = I.Dest_Preg
            (d) if (I.FU is not pipelined) then
                  Scoreboard[I.FU].Busy = False
            (e) Delete I from Sched Queue

[ Register file ]
      if (CDB.Busy = True) then
            (a) Regs[CDB.PReg].Ready = True           // Update the RF contents
            (b) Regs[CDB.PReg].Value = CDB.Value      // update the Preg
            (c) CDB.Busy = False
```


37

Pregs/RAT vs. Tomasulo

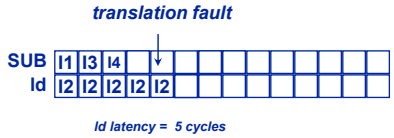
- Pregs/RAT has no associative lookup in the scheduling queue (or anywhere else), no "broadcast" to listen to
- But Tomasulo doesn't need to check the register file for all source registers, for each entry, on every cycle
- Problem: when do you free Pregs??
  - Answer in a little bit…

ILP-38

38

State Update

I1 SUB R1, R4, R5
I2 LW R2, R3(R4)
I3 SUB R3, R1, R5
I4 SUB R1, R3, R1



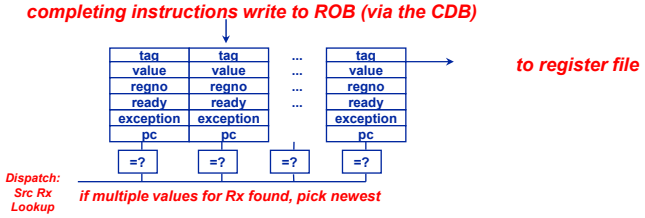
where do you restart the code after the page fault is serviced?
I3 already destroyed R3, I4 destroyed R1 so I3 cannot be re-executed

- Problem: No consistent state
  - OS needs "resume PC" to restart process after fault is handled
  - Also called: *Imprecise Interrupts*
- Definitions:
  - Fault: An excepting instruction that should be retried after exception is fixed
  - Trap: An excepting instruction that should *not* be retried after exception
  - Exception handler: routine that fixes exceptions
  - An architectural state: (1) register contents, (2) memory contents, (3) PC value

ILP-40

40

Solution #1: Reorder Buffer with Bypass (ROB w/bypass)



- Add comparators to search reorder buffer for src regs
  - The search uses source register numbers of the dispatching instruction: replaces register file lookup in basic Tomasulo
  - Search from newest to oldest ROB entry
  - If no match, get the value from the register file
  - See next slide for algorithm modifications
- Advantage: No longer have to wait until retirement for destination register to be updated
- Register file only stores the values, no additional meta data
- Problem:
  - Comparators are expensive
  - ROB now similar to fully-associative cache: a cycle time stretcher

Tomasulo & ROB w/bypass

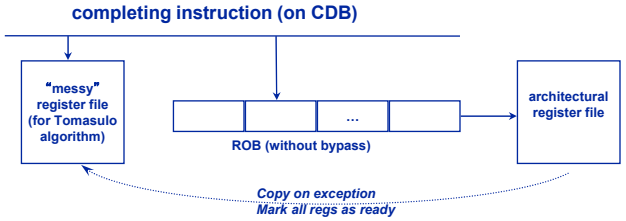
```
[Dispatch unit ] for all I in DispatchQueue do:
    if (SchedQueue is not full) then
        (a) Add I to first free slot of SchedQueue  (="RS")        // Call this RS below
        (b) Delete I from DispatchQueue                                  // Inst is now "dispatched"
        (c) RS.FU = I.FU
        (d) RS.DestReg = I.Dest
        (e) for i = 0 to Max Source Reg-1
                for j = ROB.tail to ROB.head                       // search for source registers in ROB
                    if (ROB[j].regno = RS.Src[i]) then
                            found_in_rob = True
                            if (ROB[j].Ready = True) then
                                    RS.Src[i].Value = ROB[j].Value
                                    RS.Src[i].Ready = True
                            else
                                    RS.Src[i].Tag = ROB[j].Tag  // Copy the tags from the ROB
                                    RS.Src[i].Ready = False
                            break
                if (found_in_rob = True) then continue  // go to next source reg
                else // no write of reg in ROB, get the value from the register file (because instruction retired long ago!)
                        RS.Src[i].Value = Regs[RS.Src[i]].Value
                        RS.Src[i].Ready = True

        (f) ROB[tail].Tag = unique tag id          // tag (rename) the dest, could just use the ROB index ('tail') as tag#
        (g) ROB[tail].Regno = I.Dest
        (h) ROB[tail].Ready = False
        (i) ROB[tail].Exception = False
        (j) tail++
```

Tomasulo & ROB w/bypass (cont.)

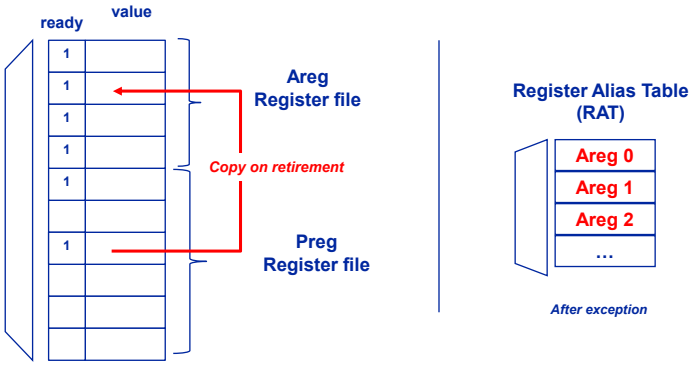
```
[At instruction completion of instruction I at index 'index' in the ROB]:
        (a) ROB[index].Value = I.Value
        (b) ROB[index].Ready= True
        (c) If exception for instruction I then
                ROB[index].Exception = true
[At instruction retire, at state update unit]:
        if (ROB[head].exception = false) then
                Regs[ROB[head].Regno].Value = ROB[head].Value
        else /* exception occurred */
                (a) resume_pc =  ROB[head].pc
                (b) tail = head // flush the ROB
                (b) call exception handler
                (c) if trap, then pc = resume_pc+4
                    else /* fault */ pc = resume_pc
```


Solution #2: ROB w/Future File



- Changes vs. ROB w/bypass are:
  - Keep the original Tomasulo register file (called "messy")
  - Completing instructions broadcast on CDB and update the messy RF as in original Tomasulo algorithm
  - Dispatch checks "messy" register file (also called *Future File* by Smith/Pleszkun in 1988) just like original Tomasulo algorithm.
  - Before exception handling, architectural register file is copied to messy register file
- The most commonly implemented scheme

ROB w/Future File: Pregs/RAT version



1. When instruction reaches head of ROB, copy Preg to corresponding Areg
2. On exception, change RAT to point to corresponding Aregs

46

Freeing PRegs via ROB

THIS IS HOW YOU FREE Pregs:

- At DISPATCH of instruction "*I*"
  - Allocate slot in ROB for *I* at tail of ROB
  - ROB[tail].Previous_preg = RAT[I.Dest_areg]

  (Rest of Dispatch is the same as prior PReg/RAT)
- On instruction retirement of *I* at head of ROB:
  - Mark ROB[head].Previous_preg as free

47

Pregs/RAT with Preg freeing

For each cycle of execution…

```
[Dispatch unit ] for all instructions I in DispQ do:
    if (SchedQ is not full AND there are free Pregs available) then
        (a) Add I to first free slot of SchedQ  (="RS")      // Call this RS (for "reservation station") below
        (b) Delete I from DispQ                               // Insttruction I is now "dispatched"
        (c) RS.FU = I.FU
        (d) for i = 0..1 do
                RS.SRC[i].reg_index = RAT[I.srci]
        (e) ROB[tail].Dest_Areg = I.Dest
        (f) ROB[tail].Prev_Preg = RAT[I.Dest]
        (g) RAT[I.Dest] = Find a free Preg #   // rename the dest
        (h) RS.Dest_Preg = RAT[I.Dest]
        (i) Regs[ RAT[I.Dest] ].Ready = False
        (j) ROB[tail].Ready = False
        (k) ROB[tail].Exception = False
        (l) tail++


    else exit loop // stop dispatching if scheduling queue is full
```

ILP-48

48

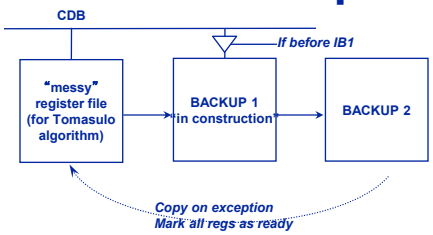
Pregs/RAT with Preg freeing

```
[At instruction retire, at state update unit]:
        if (ROB[head].exception = false) then
                (a) Regs[ROB[head].Dest_Areg].Value = ROB[head].Value
                (b) Mark ROB[head].Prev_Preg as Free in Preg free list
        else /* exception occurred */
                (a) resume_pc =  ROB[head].pc
                (b) tail = head // flush the ROB
                (b) call exception handler
                (c) for Areg = 0 to max_Areg do
                        RAT[Areg] = Areg
                (d) if trap, then pc = resume_pc+4
                    else /* fault */ pc = resume_pc
```

ILP-49

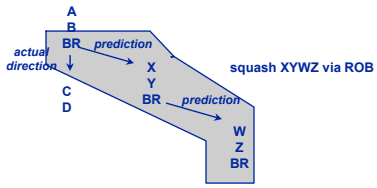
49

Solution #3: Checkpoint repair



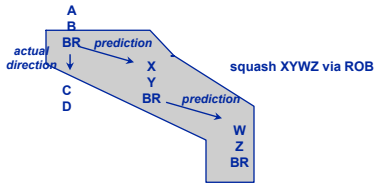
- Periodically mark an instruction barrier (IB1) at dispatch
- All instructions that complete that are before IB1 write to both messy and Backup 1
- Once all instructions before IB1 have completed, copy Backup 1 to Backup 2, Messy to Backup 1, set IB2 = IB1, and pick a new IB1
- Note that all loads and stores are buffered until backup 1 copied to backup 2
- If an exception occurs, copy Backup 2 to messy and resume at IB2 for Backup 2 after exception => may have to re-execute some instructions between IB and where the exception occurred
- Note: for a TRAP, set a new IB1 right before the TRAP and don't handle the trap until Backup 1 is complete
- Invented in 1988 by Patt and Hwu

Speculative execution



- Add branch prediction to fetch unit
- Could delay dispatch until a branch's behavior is known, OR...
- When branch is predicted, begin executing instructions speculatively until outcome is known
- Need a technique to recover from miss-speculation so speculative instructions do not hurt processor state...

Miss-speculation recovery



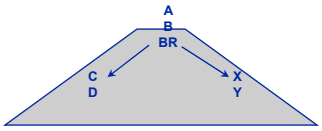
- Conceptually like "mini-exceptions"
- ROB recovery:
  - Flush all instructions from branch to tail of reorder buffer
  - No need to wait until branch is retired
  - Resume execution at correct path of branch
- Checkpoint repair recovery:
  - On branch prediction, start a new IB and copy Messy to Backup 1.
  - On mis-speculation recovery, do not write back results of any instructions after IB
  - Instead, just copy Backup 1 to Messy and resume executing instructions after the branch

ILP-52

52

Alternative to speculation: Multipath execution

- If a branch is hard to predict, execute both paths and squash one
- Downsides: always wasting space, actual path competes with squashed path for resource (FUs)



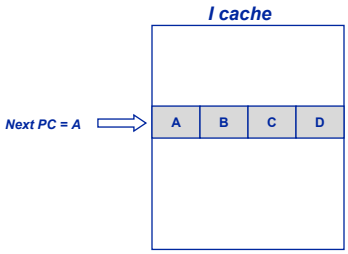
ILP-53

53

High-level Inst Fetch issues

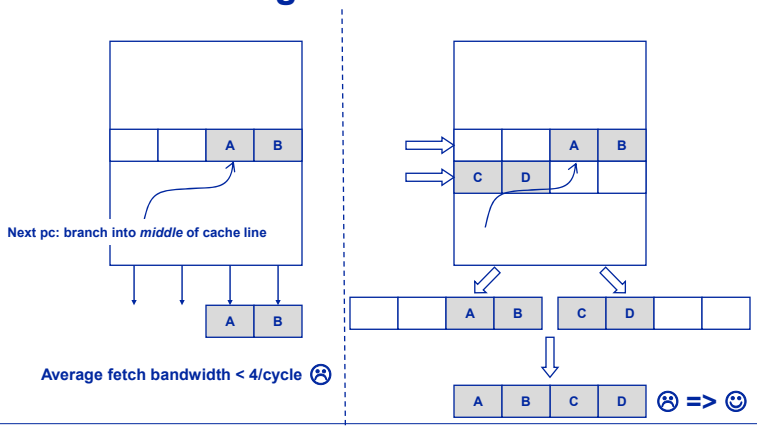
- Challenges:
  1. Need to fetch more than one instruction per cycle to keep superscalar pipeline fed
  2. I$ is single ported– can't read more than one block a cycle
  3. Branches occur every 5 inst. on average, thus need to predict branches for multiple instructions per cycle
- Solutions (not covered here):
  - Collapsing buffer: uses a banked cache and banked branch predictor plus a combining network
  - Trace cache: remembers a path through the code across multiple branches

Instruction fetch and the Icache



- Superscalar fetch needs multiple (e.g., four) instructions per cycle
- In the above, in a traditional cache, fetching A, B, C, D would take four cache accesses
- Better approach: fetch a whole block at a time from the Icache

Problem: unaligned accesses

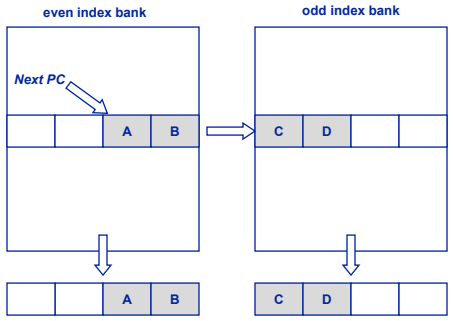


- Solution: dual-port the I$ in order to read out two *consecutive* cache lines
- Merge needed instructions together to get 4 inst./cycle bandwidth
- EXPENSIVE: true dual-porting increases I$ area significantly

ILP-56

56

"Fake" dual porting via dividing the cache into two banks

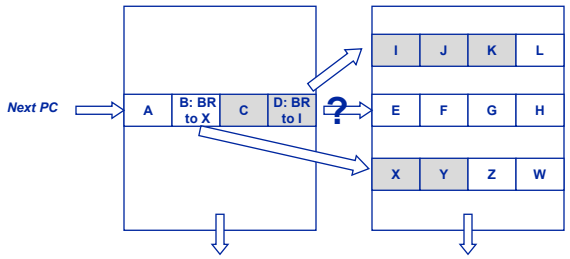


- Scheme is called "Banked Sequential"
- Select to start fetching in bank even or odd by using the least significant bit of the cache index (msb = 0: even, else odd)

ILP-57

57

Next problem: *Predicting* branches!

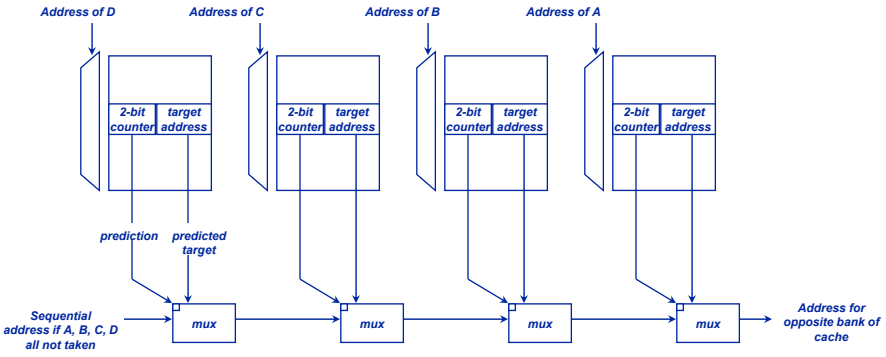


- If either B or D is (predicted) taken, want XYZW or IJKL blocks next, not EFGH block
- Also, if both B is taken and D is taken, want XYZW, not IJKL
  - because B's branch prediction has a "higher priority" than D's branch prediction

ILP-58

58

Interleaved BTB (works with any branch predictor)



- Banks of BTB accessed in parallel
- 1-cycle trip time across "carry chain"
- Overall scheme name: "Banked cache w/BTB"

ILP-59

59

Pipeline Icache Timing and Branch Penalty

Fetched cache blocks *a, b, c, d, e, f, ...*

BTB	*a*	*b*	*c*	*d*	*e*	...	...
Cache data array access		*a/b*	*b/c*	*c/d*	*d/e*	*e/f*	...
Align, tag compare			*a/b*	*b/c*	*c/d*	*d/e*	*e/f*

For simplicity, each block is fetched twice

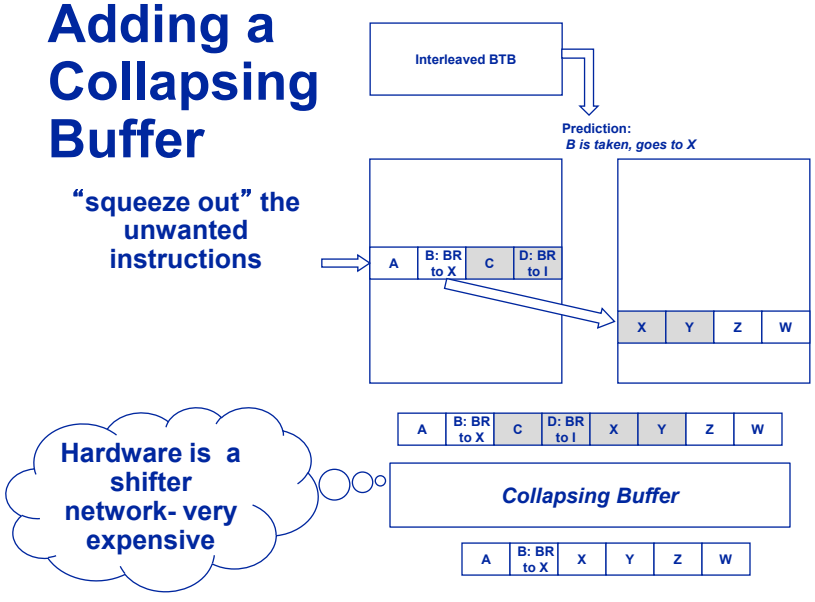
Branch prediction penalty: adds two stages to ifetch, so
2 (=Array+Align) + (stages from decode to when branch direction is resolved)

ILP-60

60

Adding a Collapsing Buffer

"squeeze out" the unwanted instructions



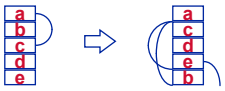
ILP-61

61

Effects of compiler optimizations on superscalar instruction fetch

- Instruction layout enhances Icache performance
  - Use instruction layout from cache miss rate reduction, e.g.,

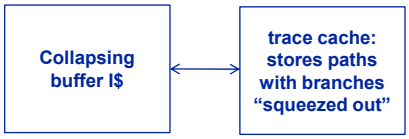
a c d e preferred path (determined by profiling)



- Does instruction layout help superscalar fetch?
- Experimental results:
  - Integer benchmarks from SPEC92
  - 5 profiling inputs, 1 (separate) test input for results
  - Net result: Instruction layout can make banked sequential perform as well as the collapsing buffer

Modification: add a "trace cache"

- Idea:



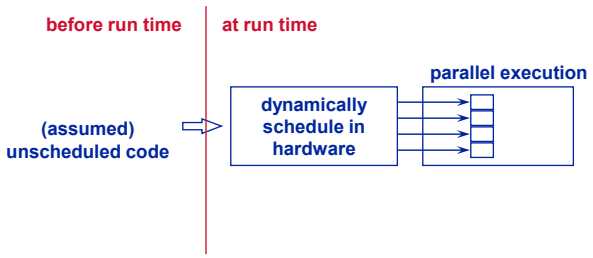
- Cache the output of the collapsing buffer as "traces"
- Look up (target of previous trace address, branch prediction) in trace $
- If a trace doesn't exist, use collapsing buffer I$ to build new trace

The limits of superscalar

- Empirically, parallelism is proportional to the $\log_2$ (*scheduling queue size*)
- Scheduling queue size is ~ to cycle time and power
- These two are conflicting goals

64

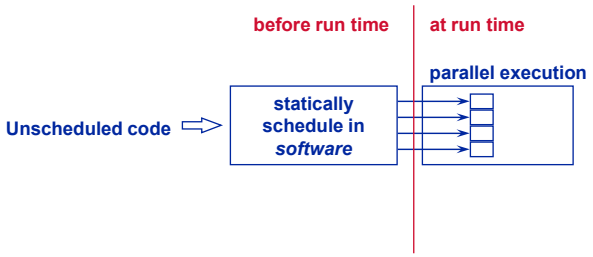
A View of Superscalar



- Impact of complexity
  - Added hardware complexity may increase cycle time
  - Added hardware complexity uses power--may exceed power envelope

65

Very Long Instruction Word



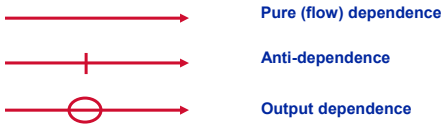
- Given that determining independence between instructions (i.e., ILP) is a problem, then *Feature it* in the ISA
- Perform all dependence checking before run-time
- Compiler plays a major role now!

ILP-66

66

Data dependence graphs

- Data structure used by compiler for scheduling
- Nodes of graph are instructions
- Arcs of graph represent dependencies
  - An arc from node X to Y means that X must execute before Y
  - Types of arcs:



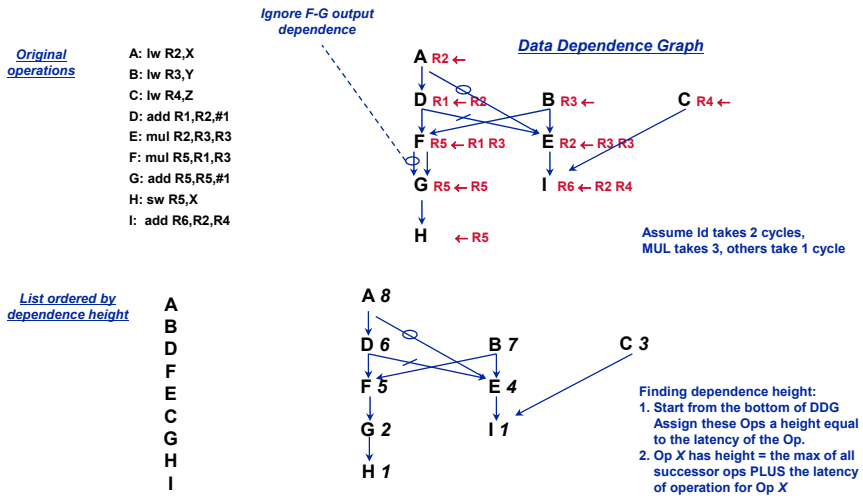
ILP-67

67

Compiler scheduling

- Optimal scheduling takes O(2^n) time steps for *n* operations worst case
- Solution: use heuristic approach
  - Most common approach is *list scheduling*
- List scheduling steps
  - (1) Build data dependence graph
  - (2) Sort graph into linear ordering (*priority list*) using heuristics (e.g., *highest-level-first*)
  - (3) Insert instructions from list into schedule subject to data dependencies and resource constraints

Example VLIW scheduling



Example VLIW machine code

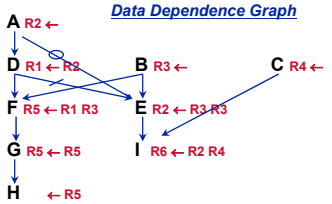
Original operations

A: lw R2,X
B: lw R3,Y
C: lw R4,Z
D: add R1,R2,#1
E: mul R2,R3,R3
F: mul R5,R1,R3
G: add R5,R5,#1
H: sw X, R5
I: add R6,R2,R4

List ordered by dependence height

A
B
D
F
E
C
G
H
I

Data Dependence Graph



IALU 1 cycle latency			MUL 3 cycle latency			Load 2 cycle latency		Store 1 cycle latency	
d	*s*	*s*	*d*	*s*	*s*	*d*	*addr*	*addr*	*s*
	nop			nop		A:R2	X		
	nop			nop		B:R3	Y		
D:R1	R2	1		nop		C:R4	Z	R2 ready	
	nop		F:R5	R1	R3			R3 ready	
	nop		E:R2	R3	R3			R4 ready	
	nop			nop				nop	
G:R5	R5	1						nop	
I:R6	R2	R4		R5 ready				H: X	R5

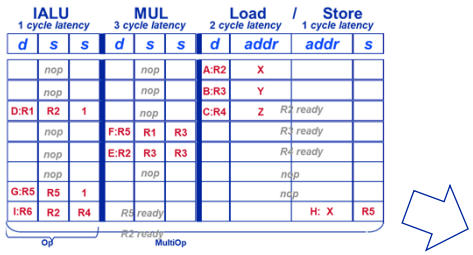
R2 ready

Op — MultiOp

Encoding the MultiOP: Getting rid of *nop*s

- When nothing to do in regular ISAs, insert a *nop*
  - But very wasteful for VLIWs!
- Horizontal *nop*s
  - Add an opcode field to every op-- encode ops as though they are RISC (eg, S3) instructions
  - Use "stop" bit-- when stop = 1, this op ends the MultiOp
  - Need a *distribution network* in hardware to send ops to their correct units
- Vertical *nop*s
  - Option 1: Insert them (they may be rare)
  - Option 2: Use "pause" field in op encoding
    - pause = *n* means wait *n* cycles before beginning the next MultiOp
    - But, every Op encoding must have a pause field, even though only its the MultiOp that requires one.
  - Option 3: Use in-order interlock "busy bits" on each register
    - If all source registers of all ops in the MultiOp are not busy, then issue the MultiOp, else issue nothing and wait a cycle
    - A "simplified" FICO scheduler…Used in IA-64
- No modern VLIW has a "stupid" encoding with *nop*s!

Example with tail and pause



Op code	Dest	Src1	Src2	Tail?	Pause
LW	R2	X	-	1	0
LW	R3	Y	-	1	0
ADD	R1	R2	#1	0	-
LW	R4	Z	-	1	0
MUL	R5	R1	R3	1	0
MUL	R2	R3	R3	1	1
ADD	R5	R5	#1	1	0
ADD	R6	R2	R4	0	-
SW	-	X	R5	1	0

VLIW semantics: parallel reads

- All sources of all ops in a multiop are read at the beginning of the cycle
- All writes to destinations happen at the end of a cycle
- Consequence: swap contents of R1 and R2

COPY R1, R2	COPY R2, R1

◆ Valid MultiOP. Invalid RISC sequence

R4 = 4
R5 = 4
R6 = 4
R7 = 6

ADD R4, R1, R3	MUL R1, R2, R3
ADD R5, R1, R3	
ADD R6, R1, R3	
ADD R7, R1, R3	

Assume MUL takes three cycles and ADD takes one
Assume R1, R2 and R3 = 2 initially

VLIW semantics: EQ vs. LTE

R4 = 4
R5 = 4
R6 = 4
R7 = 6

A: ADD R4, R1, R3	MUL R1, R2, R3
B: ADD R5, R1, R3	
C: ADD R6, R1, R3	
D: ADD R7, R1, R3	

Assume MUL takes three cycles and ADD takes one
Assume R1, R2 and R3 = 2 initially

- Writes happen later
- Two scheduler approaches:
  - EQ: All latencies equal their advertised value - compiler will schedule B, C "below" MUL (as above)– MUST have pause field, will not work with FICO busy-bit vertical NOP solution!
  - LTE: All latencies are less than or equal to their advertised value - compiler will not schedule B, C "below" MUL because MUL may complete early

Instruction scheduling

- Local vs. Global
  - Local:
    - BB code only
    - Missed opportunities because of rarely taken (not taken) branches
- Global instruction scheduling
  - Taking branches into account: *speculate* on instruction execution
- Acyclic vs. Cyclic global instruction scheduling
  - Acyclic: used for straight line code or complex loops
  - Cyclic: used for loops

Compiler plays central role

- Branch prediction?
  - Use profile data
- Instructions can be scheduled out-of-order
  - Compiler moves instructions around each other
  - Programmer's "original intent" preserved
- Speculative execution:
  - Scheduler can schedule instructions before branches
  - Instructions *execute* but results may not be *used*, depending on what the branch does
- Removing anti and output dependencies:
  - Compiler assigns registers to minimize anti-,output-dependencies

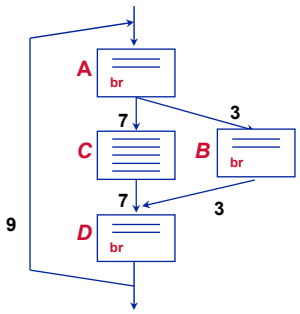
Control flow graphs & Basic Blocks

```
do {
    if (A[R5] > 0)
        A[R5] = 0;
    else
        A[R5] = A[R5]*2;
    R5--;
} while (R5 > 0);
```

A	Loop: LW R4, A(R5) BRGT R4, #0, THEN
B	ELSE: MUL R4, R4, #2 SW R4, A(R5) BR AFTER
C	THEN: SW #0, A(R5)
D	AFTER: ADD R5, R5, -1 BRGT R5, #0, Loop

***Basic block*: Longest span of sequential instructions**

***Control flow graph*: Graph of basic blocks with all possible transitions**



Global acyclic scheduling models

- Problem:
  - Parallelism in basic blocks is limited (1.5-2 empirically), causing many slots in a block's schedule to go empty
- Approach: build larger blocks of instructions before list scheduling
- Techniques:
  - Trace scheduling [Fisher]
  - Superblock Scheduling [Chen, et al.]
  - Predication/if-conversion [Allen, et al.]
  - Hyperblock Scheduling [Mahlke, et al.]
  - Treegion scheduling [Havanki, et al.]

83

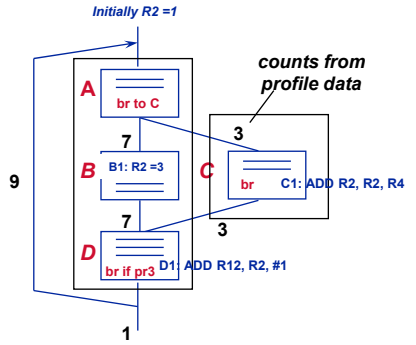
Trace scheduling

- Due to Fisher and Ellis
- Definition of "trace":
  - Linear regions of BBs in code that tend to sequentially together
  - A set of basic blocks that form a path through the code
- Approach:
  1. Form traces by following heavily weighted blocks or arcs through code
  2. Perform liveness analysis at the BB level
  3. Schedule the traces
  4. Add compensation code for side entrances and side exits using liveness information

84

Trace scheduling

	B1: R2=3		
br to C			
	D1: ADD		
			C enters here

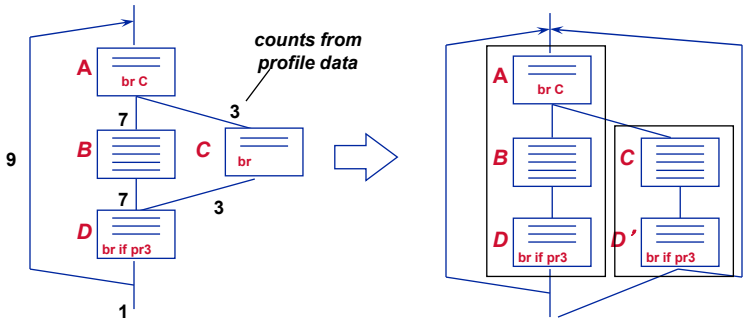


- Traces are A->B->D and C (only one block)
- Schedule A, B, D (see above, left)
- Schedule C has some complications:
  - Side exit problem: Because B1 moved before "br to C": schedule for C needs to "undo" the B1 op
  - Side entrance problem: Because D1 moved above the entrance to C, schedule for C will need a copy of D1

ILP-85

85

Superblock scheduling



- Remove all side entrance problems by duplicating D
- Larger scheduling scope for *B-D* and *C-D'*
- Still have side exit problem for instructions from B scheduled before the "br C"

ILP-86

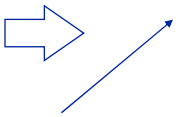
86

Predication/If-conversion

Encoding of all ops have a "predicate register number" specifier:

opcode	Dest	Src1	Src2	Predicate Reg Number

```
Loop:   LW R4, A(R5)
        BRGT R4, #0, THEN
ELSE:   MUL R4, R4, #2
        SW R4, A(R5)
        BR AFTER
THEN:   SW #0, A(R5)
AFTER:  ADD R5, R5, -1
        BRGT R5, #0, Loop
```



```
Loop:   LW R4, A(R5), P0
        CMPP.GT P1, R4, #0, P0
        NOT P2, P1
        MUL R4, R4, #2, P2
        SW R4, A(R5), P2
        SW #0, A(R5), P1
        ADD R5, R5, -1, P0
        BRGT R5, #0, Loop, P0
```

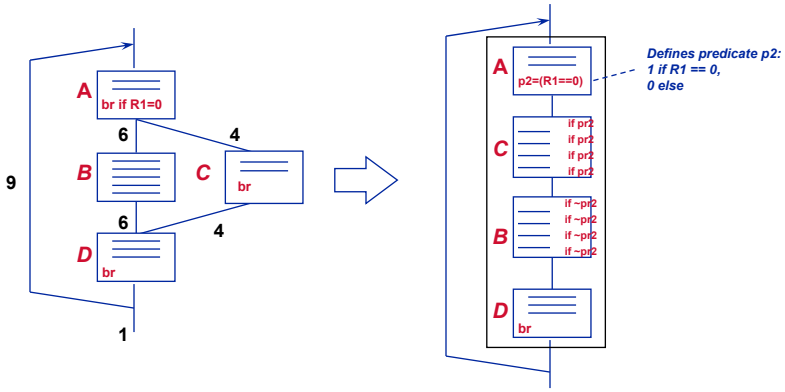
CMPP<COND> COMPARE TO PREDICATE: SETS A PREDICATE REGISTER TO RESULT OF COMPARISON <COND>

- Add 1-bit predicate registers to architecture
- All operations in a VLIW MultiOp have a predicate field
- Operations with false pred. execute as *nop*s
- Can be used to achieve multipath execution

ILP-87

87

Hyperblock scheduling



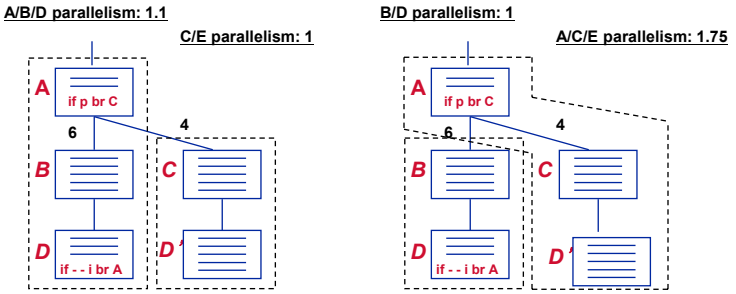
- Add 1-bit *predicate registers* to instruction format
- No branches (except for back edge), no patchup code

ILP-88

88

Problems with Linear Region Scheduling

- Close calls are dangerous:

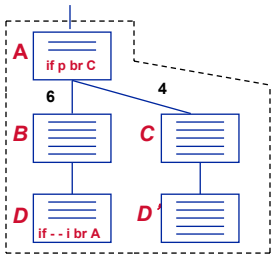


- Solutions: (1) *Use non-linear regions,* and
  (2) *Move profile usage from region formation to scheduling*

ILP-89

89

Example non-linear regions: *Treegions*

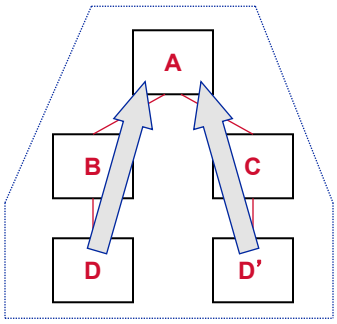


- Single-entry, multiple-exit, acyclic region
  - Each node in a treegion is a basic block
  - Related work: [Moon/Ebcioglu], [Hsu/Davidson]
- Advantages over linear regions:
  - Larger scheduling scope
  - No (or little) profile data required to form
- Limitations: No side entrances
- Implemented in NC State LeGO compiler
  - MultiOp (PlayDoh) machine model
  - Each op in a VLIW is predicate-able
  - Multi-way branches per VLIW are possible

ILP-90

90

Dominator Parallelism for Tail Duplication



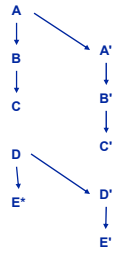
- A dominates D in original code
- Ops in D are equivalent to D'
- Ops speculated D-B-A that are identical to ops speculated D'-C-A are also *redundant*
- Treegion scheduling can remove such duplicates
- Simplified form of *Unification* PS transform from [Nicolau]

ILP-91

91

Cyclic scheduling: Loop unrolling

```
LOOP:   A: ADD R2, R2, #1
        B: LW R3, 0(R2)
        C: SW R0,0(R3)
        D: ADD R10,R10,#1
        E: BRGTZ R10,LOOP
Becomes
LOOP:   A: ADD R2, R2, #1
        B: LW R3, 0(R2)
        C: SW R0,0(R3)
        D: ADD R10,R10,#1
        E*: BRLEZ R10,EXIT
        A': ADD R2, R2, #1
        B': LW R3, 0(R2)
        C': SW R0,0(R3)
        D': ADD R10,R10,#1
        E': BRGTZ R10,LOOP
EXIT:
```



ALU	ALU	MEM	MEM
A			
A'		B	
	D	B'	
E*	D'	C	
E'		C'	

Anti-, output dep not shown!

Resources: 2 adder/branch units, 2 memory units
Add latency = 1
Memory latency = 2

ILP-98

98

Cyclic scheduling: software pipelining

Goal: Keep same schedule per iteration
Why? Saves work scheduling, where scheduling is $O(N^2)$

Initiation Interval = 1

ALU	ALU	MEM	MEM
A1			
A2		B1	
A3	D1	B2	
D2/A4	E1	B3	C1
E2	D3		C2
	E3		C3

Cannot find a same schedule for all iterations that doesn't result in a collision – just one example shown here, but this result is provable for this loop

Initiation Interval = 2

ALU	ALU	MEM	MEM
A1			
		B1	
A2	D1		
	E1	B2	C1
A3	D2		
	E2	B3	C2
A4	D3		
	E3	B4	C3
	D4		
	E4		C4

II (initiation interval) of 2 works!

"Re-roll" the loop
Note: Branch condition is modified

ALU	ALU	MEM	MEM	
A				PROLOG
	E*	B		
A	D			KERNEL
	E	B	C	
*	D			
			C	EPILOG

Note: E* uses the opposite branch test from E and branches to the Epilog (marked by *)

ILP-99

99

Iterative Modulo Scheduling

- The number of cycles between the start of each iteration is termed the *Initiation Interval (II)*
- The *Minimum Initiation Interval (MII)* is the theoretically lowest II possible
  - Two bounds to this: Resource-constrained MII (*ResMII*) and recurrence-constrained MII (*RecMII*)
  - MII = max(ResMII, RecMII)
- Core algorithm:

```
II = MII;
while (IterativeSchedule(II) != true) do
        II = II+1;
```

where IterativeSchedule() is a special version of list scheduling that tries all alternatives

ILP-100

100

Finding ResMII

- Simplistic approximation:
  - Count the most heavily used resource for the loop body
  - For us, that means the FU type that is used the most
  - Divide this by the number of resources and round up
  - Example:
    - Loop body contains 5 ADDs and 3 MULs
    - Hardware has 2 adders and 3 multipliers
      - Ceil(5/2) = 3
      - Ceil(3/3) = 1
    - Hence, ResMII = 3
- Real-world complications:
  - Opcodes map to several alternatives (e.g., shifter or multiplier can perform some multiplications)
  - Most instructions require reservation of more than a single resource (e.g., operand-,result buses)

ILP-101

101

Renaming via *Rotating Registers*

- New idea: cyclic register renaming:
- Rx[i] is *the ith iteration ago* copy of Rx
  - Rx[i] is called a "rotating register"
  - i is called the "iteration distance"
  - Branch (BR-REMAP) that decrements the iteration pointer "RRB"
  - Rewrite code before scheduling to use Rx[0]
  - Loop carried dependencies use Rx[*distance*] as a source
- New (code prior to scheduling):

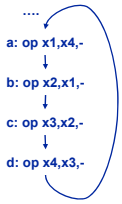
```
LOOP:   A: ADD R2[0], R2[1], 1
        B: LW R3[0], 0(R2[0])
        C: SW R0, 0(R3[0])     ; note R0 is not changed in the loop, it does not rotate
        D: ADD R10[0], R10[1], #1
        E: BRNE-REMAP R10[0], LOOP
```

ILP-102

102

Finding RecMII

- Circular dependence (recurrence) between iterations limits initiation interval



Each cycle's length = sum *latencies of ops* in the cycle

Each cycle's iteration distance = sum of *iteration distances* in the cycle

Must be true for each cycle:

iteration distance * RecMII ≥ cycle length

or, RecMII =

$\max_{i \text{ in (all cycles)}} \text{Ceil}(\text{cycle length}_i / \text{iteration distance}_i)$

ILP-103

103

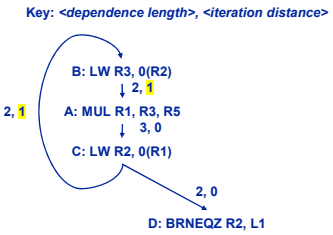
Example of iteration distance > 1

L1: A: MUL R1[0], R3[1], R5

B: LW R3[0], 0(R2[1])

C: LW R2[0], 0(R1[0])

D: BRNE-Remap R2[0], L1



For cycle B (previous iteration)=>A=>C=>B…:

Iteration distance = 1+1+0 = 2

Cycle length = 2+3+2 = 7

RecMII = Ceil(7/2) = 4

ILP-104

104

Modulo Reservation Table

- Schedule the loop in a reservation table (empty schedule) of height = II
- Say II = 3, and there are four resources, MRT is a 3x4 array
- Scheduling is then done as follows:

1. Perform list scheduling
2. Attempt to schedule an operation at all possible times ≥ (T mod II)
3. If there is a conflict, try different list ordering

ILP-105

105

Example

LOOP: A: ADD R2[0], R2[1], 1
B: LW R3[0], 0(R2[0])
C: SW R0[0],0(R3[0])
D: ADD R10[0],R10[1],#1
E: BRNE-REMAP R10[0],LOOP



II = 2

Resources: 2 adder/branch unit, 2 memory units
Add latency = 1
Memory latency = 2
ResII = max(3/2, 2/2) = 2
RecII = 1 (due to (A-A)/1 or (D-D)/1)
MinII = Max(ResII, RecII) = 2

Step 1:

ALU	ALU	MEM	MEM
A			

MRT

Step 2:

ALU	ALU	MEM	MEM
A			
		B	

Step 3:

ALU	ALU	MEM	MEM
A			
		B	C

Step 4:

ALU	ALU	MEM	MEM
A	D		
		B	C

Step 5:

ALU	ALU	MEM	MEM
A	D		
	E	B	C

Final code: A: ADD R2[0], R2[0], 1; D: ADD R10[0],R10[0],#1;;
E*: BREQ R10[0], EXIT; B: LW R3[0],0(R2[0]);;
LOOP: A: ADD R2[0], R2[1], 1; D: ADD R10[0],R10[1],#1;;
B: LW R3[0], 0(R2[0]); C: SW R0[0],0(R3[1]); E: BRne-REMAP R10[0],LOOP;;
EXIT: ;;
C: SW R0[0],0(R3[0]);;

Where ";;" means end of multiop

ILP-107

107

Problems & solutions

- Conditional code in loop body
  - Solution #1: If-conversion
  - Solution #2: Schedule one path through the loop
  - Solution #3: Schedule each path independently and then try to merge the schedules

Summary

Key: done by hardware
done by compiler

	Superscalar	VLIW
pipelined execution	IF, ID, Disp, Sched, EX, SU	IF, ID, EX, SU
Removing WAR/WAW	Tomasulo alg. or Pregs/RAT	Large register file AND Rotating register file
Resources	Scheduling queue, CDB etc.	none: sched in compiler
Speculation	Branch prediction, squashing	Predicated execution with global code motion